

IOCL GRADE-A 2026
CS/IT WORKBOOK
DAY 5 — CONCURRENCY, LOCKS, 2PL,
DEADLOCKS, FILE ORGANIZATION,
INDEXING & TIMESTAMP ORDERING
23 AUGUST 2026 (EXTENDED / GATE-COMPATIBLE EDITION)

Day 5 Objective

Day 5 completes the DBMS transaction/concurrency cluster and adds the physical-storage layer: locks, two-phase locking (2PL), deadlock conditions and detection, timestamp ordering, file organization, and indexing (dense/sparse, primary/secondary, B-tree/B+ tree, including small numericals). This day builds directly on Days 1–4’s foundation in transactions, ACID, serializability, and recoverability, and is calibrated so the same material also covers the standard GATE CS/IT DBMS question patterns on these topics.

Recommended Day 5 Workload

Block	Time	What to do
Recap Day 4	5–10 min	ACID, transaction states, serializability
Locks	30–35 min	S/X locks, compatibility, conversion
2PL	30–35 min	Basic, strict, rigorous 2PL; growing/shrinking phases
Deadlocks	25–30 min	Coffman conditions, RAG, prevention, avoidance, detection
Timestamp Ordering	15–20 min	TO rules, Thomas Write Rule (recognition + 1–2 schedules)
File Organization	15–20 min	Heap/sorted/hash/clustered, blocking factor, block-count
Indexing & Trees	20–25 min	Dense/sparse, primary/secondary, B-tree/B+ tree + numericals
MCQs	20–25 min	Guided + independent practice
Timed mini-test	15 min	12 mixed questions + error log

Total: approximately 2 hours 45–55 minutes (extended edition; trim Timestamp Ordering and File Organization to recognition-only if short on time).

Scope Boundary

Do cover: shared/exclusive locks, lock compatibility, lock conversion, two-phase locking (basic, strict, rigorous), growing and shrinking phases, Coffman conditions, resource-allocation graphs, deadlock prevention/avoidance/detection, wait-for graphs (including the multiple-instance caveat), a small Banker’s algorithm numerical, basic timestamp ordering and the Thomas Write Rule, file organization basics (heap/sorted/hash/clustered, blocking factor, block-count), dense/sparse

and primary/secondary/clustered indexing, and B-tree/B+ tree structure with small insertion and disk-access numericals.

Do not cover deeply: MVCC internals, ARIES recovery, distributed deadlocks, advanced lock-manager implementation, full B-tree/B+ tree deletion algorithms, vendor-specific indexing implementation, or complex query-cost models. SQL syntax is intentionally excluded (covered in the separate PostgreSQL/SQL workbook).

Contents

1	Learning Objectives	5
2	Day 4 Quick Recap (5–10 minutes)	5
3	Why Concurrency Control Is Needed	7
3.1	The Anomalies	7
3.2	The Solution: Concurrency Control	7
4	Locks: Shared and Exclusive	8
4.1	Shared Lock (S Lock / Read Lock)	8
4.2	Exclusive Lock (X Lock / Write Lock)	8
4.3	Lock Compatibility	8
4.4	Lock Conversion	8
5	Two-Phase Locking (2PL)	9
5.1	What Is 2PL?	9
5.2	The Two Phases	9
5.3	Why 2PL Guarantees Conflict Serializability	9
6	Variants of 2PL	10
6.1	Key Differences	10
7	Deadlocks	11
7.1	What Is a Deadlock?	11
7.2	Four Coffman Conditions	11
8	Deadlock Prevention, Avoidance, and Detection	12
8.1	Deadlock Prevention	12
8.2	Deadlock Avoidance	12
8.3	Deadlock Detection	13
8.4	Comparison Table	14
9	Timestamp Ordering (TO) and the Thomas Write Rule	15
9.1	Bookkeeping	15
9.2	Basic Timestamp-Ordering Rules	15
9.3	The Thomas Write Rule (TWR)	15
10	File Organization and Physical Storage	17
10.1	Basic Terms	17
10.2	File Organization Types	17
11	Indexing Basics	19
11.1	Why Indexes Exist	19
11.2	Index Classifications	19
12	B-Tree and B+ Tree	21
12.1	B-Tree Basics	21
12.2	B+ Tree	21
12.3	Insertion and Splitting (Recognition + Small Numerical)	21
12.4	Disk-Access (Height) Numerical	22
12.5	B-Tree vs. B+ Tree	22

13 Common Traps (Consolidated)	24
14 Guided Practice (14 Questions)	25
14.1 Lock Compatibility (3 questions)	25
14.2 Two-Phase Locking (2 questions)	25
14.3 Deadlock, Coffman Conditions and RAG (3 questions)	25
14.4 Timestamp Ordering (1 question)	25
14.5 File Organization (1 question)	26
14.6 Indexing and B+ Tree (4 questions)	26
15 Guided Practice — Full Worked Solutions	27
16 Independent Practice (31 Questions)	29
16.1 Locks (5 questions)	29
16.2 2PL (5 questions)	30
16.3 Deadlock, RAG, Banker's and TO (5 questions)	31
16.4 File Organization (5 questions)	31
16.5 Indexing (5 questions)	32
16.6 B-Tree / B+ Tree (5 questions)	33
16.7 Bonus: GATE-Style B+ Tree Insertion Problem (1 question)	33
17 Timed Mini-Test (12 Questions — 15 minutes)	34
18 Extended Practice: File Organization, Timestamp Ordering & B+ Tree Numericals (10 Questions)	36
19 Day 1–4 Cumulative Review (8 Rapid-Fire Questions)	37
20 Daily Quiz (5 Questions)	38
21 Complete Answer Key	39
21.1 Independent Practice Answers	39
21.2 Timed Mini-Test Answers	40
21.3 Extended Practice Answers	40
22 Scoring and Diagnostic Interpretation	42
23 Revision Checklist	43
24 Things I Got Wrong Today	44
25 One-Page Cheat Sheet	45

1 Learning Objectives

By the end of Day 5, you should be able to:

- Explain why concurrency control is needed in database systems.
- Distinguish shared (S) and exclusive (X) locks and use the compatibility table.
- Understand lock conversion (upgrade and downgrade).
- Explain basic two-phase locking and identify growing and shrinking phases.
- Distinguish basic 2PL, strict 2PL, and rigorous 2PL at recognition level.
- Explain why 2PL guarantees conflict serializability.
- Identify the four Coffman conditions for deadlock.
- Distinguish deadlock prevention, avoidance, and detection.
- Understand resource-allocation graphs and wait-for graphs, including why a cycle is not always sufficient for deadlock when resources have multiple instances.
- Work a small Banker's algorithm safety check.
- Explain the basic timestamp-ordering protocol and the Thomas Write Rule at question-solving level.
- Distinguish heap, sorted, hash, and clustered file organization; compute blocking factor and block count.
- Understand index purpose, dense/sparse indexing, and primary/secondary/clustered indexing correctly (not oversimplified).
- Work small B-tree/B+ tree order, insertion-split, and disk-access numericals.
- Identify common traps quickly.

2 Day 4 Quick Recap (5–10 minutes)

Before starting Day 5, answer these rapid-fire questions without rereading Day 4.

R1 [E] [ACID] Which ACID property means all-or-nothing: a transaction either fully commits or is fully rolled back?

1. (A) Atomicity (B) Consistency (C) Isolation (D) Durability

R2 [E] [Serializability] Two operations conflict if they belong to different transactions, access the same item, and:

1. (A) Both are reads (B) At least one is a write (C) Both are writes (D) One is a commit

R3 [M] [Precedence Graph] If a precedence graph for a schedule has a cycle, the schedule is:

1. (A) Serializable (B) Not conflict-serializable (C) Recoverable (D) Strict

R4 [E] [Recoverability] A schedule is recoverable if, whenever T_j reads data written by T_i :

1. (A) They run serially (B) T_i commits before T_j commits (C) No conflicts exist (D) T_i never aborts

R5 [M] [Cascadeless vs. Strict] Which property is most restrictive?

1. (A) Recoverable (B) Cascadeless (C) Strict (D) Serializable

Answers: R1=A, R2=B, R3=B, R4=B, R5=C.

3 Why Concurrency Control Is Needed

Core Problem: Multiple transactions running concurrently can interfere with each other, causing data inconsistency even if each transaction is correct in isolation.

3.1 The Anomalies

Without proper concurrency control, interleaved transactions cause:

- **Lost Update:** Two transactions read the same value, each modify it independently, and the last write overwrites the first. Net effect: one update is lost.
Example: T_1 and T_2 both read balance = 1000. T_1 adds 100 (writes 1100). T_2 subtracts 50 (writes 950). Final balance is 950, not 1050.
- **Dirty Read:** T_1 writes a value, then aborts (rolling back its write). But T_2 read the value before T_1 aborted—so T_2 read data that never existed in any committed state.
- **Unrepeatable Read:** T_1 reads value X, then later reads X again and gets a different value (because T_2 updated it in between). The same data behaves differently within one transaction.
- **Inconsistent Analysis:** T_1 reads a sum (or aggregate) of values, but before reading all values, T_2 modifies some of them, so the sum is inconsistent with any actual database state.

3.2 The Solution: Concurrency Control

Concurrency control mechanisms (locks, timestamps, MVCC) ensure that concurrent transactions do not interfere in harmful ways. The goal is to make the result of concurrent execution equivalent to some serial (non-interleaved) execution—i.e., to achieve **serializability** while still allowing **concurrency**. Locking is one family of mechanisms; timestamp ordering (Section 8) is another, used without locks at all.

4 Locks: Shared and Exclusive

4.1 Shared Lock (S Lock / Read Lock)

Purpose: Allows a transaction to read an item without modifying it.

Semantics: Multiple transactions can hold a shared lock on the same item **simultaneously**. None of them can modify the item while any S lock is held.

Use case: T_1 reads X, T_2 reads X—both can hold S on X at the same time.

4.2 Exclusive Lock (X Lock / Write Lock)

Purpose: Allows a transaction to read and modify an item. No other transaction can read or write the item while an X lock is held.

Semantics: Only one transaction can hold an X lock on an item at any time.

Use case: T_1 writes X—no other transaction can read or write X until T_1 releases the X lock.

4.3 Lock Compatibility

Two locks on the same item by different transactions are compatible (can both be held) if and only if both are shared locks:

Lock Mode	Shared (S)	Exclusive (X)
Shared (S)	Yes	No
Exclusive (X)	No	No

Key insight: S–S is compatible; everything else is not. A conflicting *request* does not abort the requester by default — it normally **waits** in the lock queue until the conflicting lock is released.

4.4 Lock Conversion

A transaction may convert its lock on an item:

- **Upgrade:** $S \rightarrow X$ (request exclusive lock while holding shared lock).
- **Downgrade:** $X \rightarrow S$ (demote to shared lock while holding exclusive lock).

Upgrades can create deadlock risk (covered later). Downgrades are less problematic.

Common Trap

Trap 1: Assuming S–X are incompatible in a symmetric “either way” sense that permits coexistence. They are not — the compatibility rule is precise: S–S is allowed; everything else (S–X, X–S, X–X) is not allowed.

Trap 2: Thinking locks alone prevent all anomalies. Locks must be used **in protocol** (like 2PL) to guarantee serializability.

Trap 3: Confusing “T2 cannot acquire X while T1 holds S” with “T2 is aborted.” By default T2 simply **waits**; abort only happens under specific deadlock-handling policies (e.g., wait-die, wound-wait) or explicit timeout.

5 Two-Phase Locking (2PL)

5.1 What Is 2PL?

Definition: A transaction obeys two-phase locking if it acquires locks in a **growing phase** and releases them in a **shrinking phase**, with no interleaving of acquisitions and releases.

5.2 The Two Phases

- **Growing Phase:** The transaction acquires locks (S or X) as needed. No locks are released during this phase.
- **Shrinking Phase:** Once the transaction releases its first lock, it enters the shrinking phase. In this phase, it may only release locks—no new locks can be acquired.
- **Lock Point:** The moment when the transaction has acquired all the locks it needs (end of growing phase, start of shrinking phase).

5.3 Why 2PL Guarantees Conflict Serializability

If all transactions follow 2PL, the precedence graph (from conflicting operations) is guaranteed to be acyclic, which means the schedule is conflict-serializable. This is a key theorem.

Simple 2PL Example

Transaction T_1 (follows 2PL):

1. Lock S on X
2. Read X
3. Lock X on Y
4. Write Y
5. Release lock on X (enter shrinking phase)
6. Release lock on Y
7. Commit

Growing phase: steps 1–4 (acquire S on X, then X on Y). Shrinking phase: steps 5–7 (release X on X, then Y on Y).

Common Trap

Trap 1: Thinking 2PL means all locks are held until commit. That's **strict** or **rigorous** 2PL (see next section), not basic 2PL. In basic 2PL, locks can be released during the transaction as long as no new locks are acquired afterward.

Trap 2: Confusing 2PL with deadlock prevention. 2PL does NOT prevent deadlocks—it only ensures conflict serializability.

6 Variants of 2PL

All three variants enforce the growing/shrinking phase rule, but differ in when locks are released:

Variant	Lock Release	Effect on Schedules
Basic 2PL	Any lock can be released during or after the transaction's execution; must obey growing/shrinking constraint.	Guarantees conflict serializability; may allow cascading roll-back.
Strict 2PL	All X locks held until commit/abort. S locks can be released earlier.	Guarantees conflict serializability, cascadeless schedules, and recoverability (no dirty writes are ever read).
Rigorous 2PL	All S and X locks held until commit/abort.	Guarantees conflict serializability, cascadeless, and strict schedules. Simplest recovery; transaction commit order equals the equivalent serial order.

6.1 Key Differences

- **Basic 2PL:** Minimum locking constraint. Provides serializability but not cascadeless.
- **Strict 2PL:** X locks held until end. Provides serializability, cascadeless schedules, and recoverability, but allows early S release. **This is the variant most real database systems implement.**
- **Rigorous 2PL:** All locks held until end. Simplest, strongest guarantee, but more lock-holding overhead.

Common Trap

Trap: Confusing strict 2PL with the **strict schedule** concept from Day 4. Strict 2PL is a locking protocol; strict schedule is a schedule property. Strict 2PL guarantees cascadeless and recoverable schedules; it is **rigorous** 2PL that is specifically associated with strict schedules in the way commit order is treated. They are related but not identical — read the exact statement carefully in an MCQ.

7 Deadlocks

7.1 What Is a Deadlock?

A deadlock is a situation where two or more transactions are waiting for each other to release locks, resulting in a circular wait. No transaction can proceed.

Simple Deadlock

T1: Lock X, then request lock on Y (blocked, waiting for T2).

T2: Lock Y, then request lock on X (blocked, waiting for T1).

Result: Circular wait. Both transactions are blocked indefinitely.

7.2 Four Coffman Conditions

The four Coffman conditions are **necessary** conditions for deadlock — all four must hold for a deadlock to exist, and eliminating **any one** of them makes deadlock impossible:

1. **Mutual Exclusion:** Each resource (lock) can be held by at most one transaction at a time.
2. **Hold and Wait:** A transaction holding at least one lock may request another lock (while holding the first).
3. **No Preemption:** No lock can be forcibly taken away; only the holding transaction can release it.
4. **Circular Wait:** A cycle of transactions exists where each is waiting for a lock held by the next in the cycle.

Key insight: If **any one** of these conditions is eliminated, deadlock cannot occur.

Common Trap

Trap: Memorizing only 3 of the 4 conditions. All four must be present for deadlock to occur. Make sure you know all four.

8 Deadlock Prevention, Avoidance, and Detection

8.1 Deadlock Prevention

Idea: Eliminate at least one Coffman condition **permanently** by system design.

- **Eliminate Mutual Exclusion:** Not practical (resources like database items must be exclusive).
- **Eliminate Hold and Wait:** Require transactions to request all locks upfront before executing, or use hierarchical/ordered lock acquisition (e.g., resource ordering).
- **Eliminate No Preemption:** Allow lock preemption (forcibly take locks from low-priority transactions). E.g., **wound-wait** and **wait-die** (timestamp-based schemes; see below).
- **Eliminate Circular Wait:** Enforce a total ordering on all resources; transactions must request locks in increasing order of resource ID.

Cost: May reduce concurrency by forcing unnecessary lock acquisition or creating overhead.

Wait-Die and Wound-Wait (Recognition Level)

Both schemes assign each transaction a timestamp at start (older transaction = smaller timestamp) and use it to decide what happens when T_i requests a lock held by T_j :

- **Wait-Die** (non-preemptive): if T_i is **older** than T_j , T_i waits. If T_i is **younger**, T_i dies (aborts and restarts with the same timestamp later).
- **Wound-Wait** (preemptive): if T_i is **older** than T_j , T_i **wounds** T_j (forces T_j to abort). If T_i is **younger**, T_i waits.

Memory aid: in both schemes, an **older** transaction is never made to abort by a younger one. Wait-die makes the *younger requester* abort; wound-wait makes the *younger lock-holder* abort. Both schemes are provably deadlock-free because timestamps only ever decrease along any wait chain, so a cycle cannot form.

8.2 Deadlock Avoidance

Idea: Before granting a lock, check if granting it would move the system into an **unsafe state** (one from which deadlock is possible). If so, delay the request.

- **Safe State:** One from which the system can always reach completion (no deadlock) via *some* order of finishing transactions.
- **Unsafe State:** One where deadlock is possible (not certain).
- **Banker's Algorithm:** A classic avoidance algorithm. Before granting a resource request, simulate allocation and check if a safe sequence of transaction completion exists.

Key Insight: Unsafe state does **not guarantee** deadlock—deadlock is only **possible**. Prevention eliminates it **for certain**; avoidance is a middle ground.

Practical Note: Banker's algorithm requires knowing the maximum resources each transaction needs upfront, which is impractical for database systems. Avoidance is used in some

embedded/OS resource-management settings and is tested in GATE mainly as an OS-flavoured numerical; it is rarely used in production databases.

Small Banker's Algorithm Safety Check

Three transactions T_1, T_2, T_3 compete for a single resource type with **10** total instances.

Transaction	Allocation	Max	Need (Max – Allocation)
T_1	3	7	4
T_2	2	4	2
T_3	2	5	3

Total allocated = $3 + 2 + 2 = 7$. Available = $10 - 7 = 3$.

Check: Can any transaction finish with Available = 3?

- T_2 's Need = $2 \leq 3$. Run T_2 to completion; it releases its resources: Available = $3 + 2 = 5$.
- T_3 's Need = $3 \leq 5$. Run T_3 ; releases resources: Available = $5 + 2 = 7$.
- T_1 's Need = $4 \leq 7$. Run T_1 ; releases resources: Available = $7 + 3 = 10$.

A safe sequence $\langle T_2, T_3, T_1 \rangle$ exists, so the state is **safe**. If instead Available had been, say, 1 (with the same Allocation), no transaction's Need would be ≤ 1 , so the state would be **unsafe** — deadlock would be *possible* but not yet certain.

8.3 Deadlock Detection

Idea: Allow deadlock to happen, then detect it and recover by aborting one or more transactions.

- **Resource-Allocation Graph (RAG):** A more general graph with two node types — transactions and resources. A **request edge** $T_i \rightarrow R_j$ means T_i is waiting for resource R_j ; an **assignment edge** $R_j \rightarrow T_i$ means R_j is currently held by T_i .
- **Wait-for Graph:** A simplified RAG used when every resource (lock) has exactly **one instance**. Nodes are transactions only; edge $T_i \rightarrow T_j$ means T_i is waiting for a lock held by T_j (obtained by collapsing resource nodes out of the RAG).
- **Deadlock Indicator:** For the **single-instance-per-resource** model (the standard DBMS locking case), a cycle in the wait-for graph is both **necessary and sufficient** for deadlock.
- **Recovery:** Abort one or more transactions in the cycle to break the deadlock. The aborted transactions release their locks.

Wait-for Graph Example

Suppose:

- T_1 holds X, waits for Y (held by T_2).
- T_2 holds Y, waits for Z (held by T_3).
- T_3 holds Z, waits for X (held by T_1).

Wait-for graph: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ (a cycle).

Conclusion: Deadlock exists (each resource X, Y, Z has a single instance here). Abort one transaction (e.g., T_3) to break the cycle.

Common Trap

Trap 1: Confusing wait-for graph (transactions and waits) with precedence graph (transactions and conflicting operations) from Day 4. Different concepts, different uses.

Trap 2: Thinking detection is “better” than prevention/avoidance. Detection wastes resources on aborted transactions; prevention/avoidance incurs overhead upfront. The right choice depends on expected deadlock frequency.

Trap 3 (GATE-style, important): “A cycle in the wait-for graph always implies deadlock” — true **only when every resource type has a single instance** (the normal DBMS lock case). If a resource type can have **multiple instances** (as in general resource-allocation-graph / OS-style problems), a cycle is **necessary but not sufficient** for deadlock: the cycle might still resolve if enough instances become free along the way. Read the question carefully to see whether it says “single instance of each resource” or gives multiple instances of a resource type.

8.4 Comparison Table

Approach	Prevention	Avoidance	Detection
When applied	Before deadlock	Before deadlock	After deadlock
Cost	Lock/ordering overhead	Simulation overhead	Abort/redo overhead
Example	Resource ordering, wait-die/wound-wait	Banker’s algorithm	Wait-for graph / RA
Concurrency	Reduced	Good	Best
Practical use	Yes (mostly)	Rare in DBs	Common in DBs

9 Timestamp Ordering (TO) and the Thomas Write Rule

Timestamp ordering is a **lock-free** alternative to 2PL for guaranteeing serializability: instead of locks, each transaction T is given a unique timestamp $TS(T)$ when it starts (older transaction $\Rightarrow$ smaller timestamp). The protocol enforces that conflicting operations execute in timestamp order.

9.1 Bookkeeping

For every data item X , the system tracks:

- $read_TS(X)$: the largest timestamp among transactions that have successfully read X .
- $write_TS(X)$: the largest timestamp among transactions that have successfully written X .

9.2 Basic Timestamp-Ordering Rules

Read rule: When T requests to read X :

- If $TS(T) < write_TS(X)$: T is trying to read a value that was already overwritten by a “future” (later-timestamped) transaction. **Reject and roll back** T (restart with a new timestamp).
- Otherwise: allow the read; set $read_TS(X) = \max(read_TS(X), TS(T))$.

Write rule (basic TO): When T requests to write X :

- If $TS(T) < read_TS(X)$ **or** $TS(T) < write_TS(X)$: a later transaction has already read or written X ; writing now would violate timestamp order. **Reject and roll back** T .
- Otherwise: perform the write; set $write_TS(X) = TS(T)$.

9.3 The Thomas Write Rule (TWR)

TWR is a modification of the **basic TO write rule** that avoids some unnecessary rollbacks:

- If $TS(T) < read_TS(X)$: still **roll back** T (a later transaction already read the item; this cannot be fixed).
- If $TS(T) < write_TS(X)$: instead of rolling back, **simply ignore/skip** T ’s write (it is an “obsolete write” since a later transaction has already overwritten X with a more recent value that must stand). T is allowed to continue.
- Otherwise: perform the write as usual.

Why TWR Matters

Basic timestamp ordering guarantees **conflict serializable** schedules. The Thomas Write Rule guarantees only **view serializable** schedules (it deliberately allows a specific class of “blind writes” to be silently discarded rather than rolled back) — but it improves performance by avoiding unnecessary aborts. This exact conflict-serializable-vs-view-serializable distinction between plain TO and TWR is a classic recognition-level exam point.

Common Trap

Trap 1: Thinking TO uses locks. It does not — it is entirely timestamp-based; there is no waiting on a lock queue, only immediate accept/reject decisions.

Trap 2: Forgetting that TWR changes only the **write** rule, not the read rule. The read rule (reject if $TS(T) < write_TS(X)$) is unchanged.

Trap 3: Assuming TO and 2PL give identical schedules. Both guarantee serializability, but by different means, and the specific interleavings each one permits or rejects can differ.

6.2 A Fourth Variant: Conservative 2PL

Addendum — Section 6 named three variants; here is the fourth one occasionally asked about.

Basic, Strict, and Rigorous 2PL all differ in *when locks are released*. Conservative 2PL (also called static 2PL) differs in *when locks are acquired* instead: the transaction must acquire **all** the locks it will ever need, all at once, before it executes its first operation. If even one required lock is unavailable, the transaction acquires none of them and waits.

This trades flexibility for a strong guarantee: since a transaction never asks for a new lock mid-execution, it can never end up waiting for a resource held by another transaction that is in turn waiting on it —

Conservative 2PL is deadlock-free by construction, unlike the other three variants, which all still need external deadlock prevention/detection (Day 5 §7–8).

COMMON TRAP

The catch, and the usual MCQ angle: Conservative 2PL requires knowing a transaction's *entire* read/write set in advance, before it runs — rarely realistic for interactive or dynamically-branching transactions. This is why it's deadlock-free in theory but rarely used in practice, versus Strict 2PL which is the actual industry default.

10 File Organization and Physical Storage

Before indexing makes sense, you need to know how records are physically packed into disk blocks.

10.1 Basic Terms

- **Block (page):** the unit of disk I/O transfer, size B bytes.
- **Record:** one row of a table, size R bytes (fixed-length, for simplicity here).
- **Blocking factor bfr :** number of records that fit in one block, $bfr = \left\lfloor \frac{B}{R} \right\rfloor$.
- **Number of blocks** needed for a file of r records: $b = \left\lceil \frac{r}{bfr} \right\rceil$.
- **Spanned records:** a record is allowed to be split across two blocks (used when R is close to or larger than B).
- **Unspanned records:** a record must fit entirely within a single block (some space may be wasted at the end of a block); this is the default assumption unless a question says otherwise.

Blocking Factor / Block Count Example

Block size $B = 2048$ bytes, record size $R = 100$ bytes, number of records $r = 30,000$, unspanned organization.

$bfr = \lfloor 2048/100 \rfloor = 20$ records per block.

$b = \lceil 30000/20 \rceil = 1500$ blocks.

If the question additionally asks for a **primary index** (one index entry per block, i.e. sparse, on an ordering key) with an index-entry size of 12 bytes, the index blocking factor is $bfr_i = \lfloor 2048/12 \rfloor = 170$ entries per block, needing $\lceil 1500/170 \rceil = 9$ index blocks. A binary search over the 9 index blocks plus one data-block access is the typical way such block-access-count questions are solved: $\lceil \log_2 9 \rceil + 1 = 4 + 1 = 5$ block accesses.

10.2 File Organization Types

- **Heap (unordered) file:** new records are appended at the end. Insertion is $O(1)$ block access; search/scan requires reading (on average) half the file, and a full scan reads every block. No ordering assumed.
- **Sorted (sequential) file:** records are physically ordered by some key. Search by that key can use **binary search** on blocks: $O(\log_2 b)$ block accesses. Insertion/deletion is expensive (must maintain order, often via overflow areas).
- **Hash file organization:** a hash function maps a key value to a block address (bucket). Very fast for exact-match/equality search ($O(1)$ average), but poor for range queries and ordered scans.
- **Clustered file organization:** records are physically stored in an order or grouping chosen to improve locality for common access patterns, often based on a clustering field (this is a physical-storage technique, distinct from the “clustered index” terminology used later in this section).

Common Trap

Trap 1: Assuming a sorted file is always I/O-efficient for a **full table scan**. For a plain sequential scan (reading every record once), a **heap** file is just as I/O-efficient as a sorted file — both require reading every block exactly once; sorting only helps **search/range** operations, not a full unordered scan.

Trap 2: Thinking hash organization is good for range queries. Hashing scrambles key order, so it is excellent for equality lookups but poor for “find all records between X and Y ” — use a sorted file or a B+ tree index for that.

Trap 3: Forgetting to apply the ceiling function ($\lceil \cdot \rceil$) when computing number of blocks — a fractional block still needs a whole extra block.

11 Indexing Basics

11.1 Why Indexes Exist

Indexes speed up data retrieval by providing alternative access paths to records, reducing the need for full table scans. The trade-off: extra storage and update overhead (every insert/update/delete on an indexed column must also update the index).

11.2 Index Classifications

Dense vs. Sparse Index

- **Dense Index:** Contains an entry for **every** search-key value in the file (in a keyed dense index) or every record. Faster lookups (index alone can sometimes answer the query) but uses more storage.
- **Sparse Index:** Contains entries for only **some** records (typically one entry per block, pointing to the first/anchor record of that block). Uses less storage but requires an additional linear/scan step within the located block. Sparse indexes are only possible on an **ordered (sorted)** data file.

Primary vs. Secondary Index — Correct Definition

These terms refer to whether the index key is the **ordering key** of the file, **not** simply “is it the primary key.”

- **Primary Index:** An index built on the field that also determines the **physical ordering** of the data file (the ordering/search key). It is usually — but not necessarily by definition — the primary key. A primary index is typically **sparse** (one entry per block) because the data file is already sorted by that key.
- **Secondary Index:** An index built on a field that does **not** determine the physical order of the file (a *non-ordering* field). It can be built on a candidate key (unique secondary index) or a non-key attribute (may return multiple records). A secondary index must be **dense**, because the data is not physically sorted by that field — the index needs an entry for every value to be able to locate the record at all.
- **Clustering Index:** A special case — built on a non-key **ordering** field of a sorted file (i.e., the file is sorted by a non-key attribute). Like a primary index it is typically sparse, but unlike a primary index the ordering field is not unique.

Clustered vs. Non-Clustered Index

- **Clustered Index:** The physical order of records in the file matches the index order. There can be at most one clustered index per table. Often built on the primary key, but this is a design choice, not a requirement.
- **Non-Clustered Index:** Does not dictate the physical order of records. Multiple non-clustered indexes can exist on a table.

Common Trap

Trap 1: Equating “primary index” with “index on the primary key” and “secondary index” with “index on a non-key attribute.” The correct distinguishing factor is whether the indexed field **orders the data file**, not whether it is a key. A unique index on a non-ordering key is still a **secondary** index.

Trap 2: Confusing clustered index with B+ tree. B+ tree is a data structure; clustered index is an organizational property. A B+ tree can be used to implement either a clustered or a non-clustered index.

Trap 3: Assuming a sparse index is possible on an unsorted (heap) file. Sparse indexing requires the data file to be sorted on the index key; otherwise you cannot use “one entry per block” to reliably locate the record’s block.

12 B-Tree and B+ Tree

12.1 B-Tree Basics

A B-tree is a balanced multiway search tree optimized for disk-based storage.

- **Balanced:** All leaves are at the same depth, ensuring logarithmic-time operations.
- **Multiway:** A B-tree of **order** m : every node has at most m children and at most $m - 1$ keys; every non-root, non-leaf node has at least $\lceil m/2 \rceil$ children (at least $\lceil m/2 \rceil - 1$ keys). The root has at least 2 children (unless it is a leaf).
- **Search Efficiency:** Height is $O(\log_{\lceil m/2 \rceil} n)$, where n is the number of records — this is why B-trees stay shallow even for millions of records, keeping disk I/O low.

Structure: each node contains keys and pointers (to child nodes or, in a plain B-tree, directly to data records); keys separate the search ranges; internal nodes guide the search and **also** store data pointers/entries in a plain B-tree.

12.2 B+ Tree

B+ tree is a variant of B-tree optimized for databases:

- **Internal nodes:** Contain only keys and pointers to children—no data entries.
- **Leaf nodes:** Contain actual data entries (or pointers to records) and are linked (usually as a singly or doubly linked list) for efficient range scans.
- **Range scans:** Efficient because leaf nodes are linked—can traverse all records in a range without jumping back to internal nodes.

12.3 Insertion and Splitting (Recognition + Small Numerical)

1. Find the correct leaf for the new key (by descending the tree, as in a search).
2. Insert the key in sorted position within that leaf.
3. If the leaf now has **too many** keys (overflow, more than the maximum allowed), **split** it into two leaves at the median key. In a B+ tree, the median key is **copied up** to the parent (it also stays in the right leaf, since leaves must hold all data entries); in a plain B-tree, the median key is **moved up** (removed from the leaf).
4. Insert the copied/moved-up median key into the parent. If the parent now overflows, split the parent the same way, propagating upward.
5. If the **root** splits, a new root is created with one key and two children — this is the **only** way the tree's height increases, and it increases uniformly for every leaf (the tree stays balanced).

B+ Tree Insertion / Split Numerical

A B+ tree leaf node can hold at most 3 keys (order $p = 4$ for leaves, i.e. max 3 data entries per leaf before it must split). A leaf currently holds keys $\{10, 20, 30\}$ and is full.

Insert key 25:

1. Key 25 belongs in this leaf (between 20 and 30); inserting gives $\{10, 20, 25, 30\}$ — 4 keys, which overflows the max of 3.
2. Split at the median. With 4 keys, the split point (for a leaf, keep the ceiling in the left half by common convention) gives left leaf $\{10, 20\}$, right leaf $\{25, 30\}$.
3. The right leaf's first key, **25**, is copied up to the parent as the new separator between the two leaves.
4. If the parent (an internal node) had room, insertion stops here — **tree height does not increase**, only this leaf split into two leaves at the same depth.
5. Tree height increases **only** if this split propagates all the way up and the root itself must split.

12.4 Disk-Access (Height) Numerical

B+ Tree Disk Access Example

A B+ tree index has a fan-out (max children per internal node) of $p = 100$ and needs to index $n = 1,000,000$ records, one entry per leaf slot, with a leaf capacity comparable to the fan-out.

Approximate height $h \approx \lceil \log_{100} 1,000,000 \rceil = 3$ (since $100^3 = 1,000,000$).

A point search therefore costs about $h+1 = 4$ **block accesses**: 3 internal-node reads (root $\rightarrow$ level 1 $\rightarrow$ level 2) plus 1 leaf read. (Exact numbers vary slightly with how a question defines “order” and whether the root is counted separately — always re-derive from the given fan-out rather than memorizing a fixed height.)

12.5 B-Tree vs. B+ Tree

Feature	B-Tree	B+ Tree
Data entries	In internal and leaf nodes	Only in leaf nodes
Internal node size	Can be large (has data)	Smaller (keys + pointers only)
Range scans	Requires jumping between nodes	Efficient (linked leaves)
Space usage	More space per internal node	Better balance, higher fan-out
Database use	Less common	Standard for indexing

Common Trap

Trap 1: Assuming B-tree is always better. B+ tree is superior for databases because of efficient range scans, linked leaves, and higher fan-out (smaller internal nodes fit more pointers per block, reducing tree height).

Trap 2: Thinking B+ tree is just a B-tree with extra links. The data placement differs fundamentally — B+ tree internal nodes carry **no** data entries at all.

Trap 3: Assuming every insertion increases the tree's height. Most insertions only split a leaf (or a leaf plus a few ancestors); height increases **only** when the split propagates all the way to the root.

Trap 4: Misreading “order” — some textbooks define order as max children, others as max keys. Always check the exact definition given in the question before computing min/max bounds.

13 Common Traps (Consolidated)

Explicitly cover these in exam preparation:

1. **S–S compatibility:** S locks on the same item by different transactions are compatible (allowed). This is **not** a conflict.
2. **X conflicts with S and X:** X lock on an item is incompatible with any other lock (S or X) by a different transaction — the conflicting requester **waits**, it is not automatically aborted.
3. **2PL does not prevent deadlocks:** 2PL only guarantees conflict serializability. Deadlock prevention/avoidance/detection is separate.
4. **Basic 2PL $\neq$ Strict 2PL:** Basic 2PL allows locks to be released early (during the transaction). Strict 2PL holds X locks until commit.
5. **Unsafe state $\neq$ Deadlock:** In avoidance, unsafe state means deadlock is possible, not certain.
6. **Wait-for graph $\neq$ Precedence graph:** Wait-for graph (cycles indicate deadlock in the single-instance model); precedence graph (cycles indicate non-serializability). Different use cases.
7. **Cycle in wait-for graph is deadlock only for single-instance resources:** with multiple instances per resource type, a cycle is necessary but not sufficient.
8. **Clustered index $\neq$ B+ tree:** Clustered is an organizational property; B+ tree is a data structure.
9. **Primary index $\neq$ “index on primary key”:** the defining property is whether the field **orders** the file, not whether it is a key.
10. **Tight coupling assumption:** Indexes improve some queries but slow updates. Not every attribute should be indexed.
11. **Four Coffman conditions:** All must be true for deadlock. Removing just one prevents it.
12. **Prevention vs. avoidance vs. detection:** Prevention is permanent (system design); avoidance checks state before granting; detection runs after a cycle is suspected.
13. **Basic TO vs. Thomas Write Rule:** Basic TO guarantees conflict serializability with rollbacks on stale writes; TWR relaxes the write rule to *ignore* (not roll back) an obsolete write, guaranteeing only view serializability.
14. **Heap vs. sorted file for a full scan:** both are equally I/O-efficient for a plain sequential scan; sorting only helps search/range operations, not scan-everything queries.
15. **B+ tree height increases only on root split:** most insertions only affect a leaf (and maybe a few ancestors), not the whole tree’s height.

14 Guided Practice (14 Questions)

Attempt each question, read the hint if stuck, then check the full worked solution section before moving to Independent Practice.

14.1 Lock Compatibility (3 questions)

G1 [E] [Locks] [Concept] Can two transactions simultaneously hold shared locks on the same data item?

Hint: Consult the S-X compatibility table. Is S-S allowed?

G2 [M] [Locks] [Scenario] Transaction T_1 holds an exclusive lock on item X. Can transaction T_2 acquire a shared lock on X immediately?

Hint: What does the X row of the compatibility table say about compatibility with S? What happens to T_2 's request instead of being granted?

G3 [E] [Locks] [Conversion] A transaction holds a shared lock on item Y. Can it upgrade to an exclusive lock on Y (while still accessing Y)?

Hint: Lock conversion is a standard operation; upgrading $S \rightarrow X$ is allowed.

14.2 Two-Phase Locking (2 questions)

G4 [M] [2PL] [Scenario] A transaction acquires locks on items A, B, and C (in that order), then releases the lock on A. Can it now acquire a new lock on D?

Hint: Once a transaction releases its first lock, which phase has it entered?

G5 [M] [2PL] [Comparison] Explain why strict 2PL prevents cascading rollback while basic 2PL does not.

Hint: When are X locks released in each variant?

14.3 Deadlock, Coffman Conditions and RAG (3 questions)

G6 [E] [Deadlock] [Definition] List the four Coffman conditions for deadlock.

Hint: Mutual exclusion, hold and wait, no preemption, circular wait.

G7 [M] [Deadlock] [Detection] Draw a wait-for graph for the following scenario and determine if a deadlock exists (assume each resource has a single instance):

- T_1 holds lock on X, waits for lock on Y (held by T_2).
- T_2 holds lock on Y, waits for lock on Z (held by T_3).
- T_3 holds lock on Z, waits for lock on X (held by T_1).

Hint: Draw $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$. Is there a cycle?

G8 [H] [Deadlock] [Banker's] Using the Banker's algorithm numerical from Section 7.2 (Allocation: $T_1=3, T_2=2, T_3=2$; Max: $T_1=7, T_2=4, T_3=5$; total instances = 10), is the state safe? Give a safe sequence if one exists.

Hint: Compute Available, then check whose Need fits first.

14.4 Timestamp Ordering (1 question)

G9 [M] [TO] [Concept] Under the Thomas Write Rule, if $TS(T) < write_TS(X)$, what happens to T 's write — and how does this differ from basic timestamp ordering?

Hint: TWR “ignores” rather than “rolls back” this specific case.

14.5 File Organization (1 question)

G10 [E] [File Org] [Numerical] A block holds 2048 bytes; each record is 128 bytes; the file has 5000 records; organization is unspanned. Find the blocking factor and number of blocks.

Hint: $bfr = \lfloor B/R \rfloor$; $b = \lceil r/bfr \rceil$.

14.6 Indexing and B+ Tree (4 questions)

G11 [E] [Indexing] [Concept] Why is a B+ tree preferred over a B-tree for database indexing?

Hint: Think about range scans and linked leaves.

G12 [M] [Index] [Scenario] Distinguish between a clustered index and a non-clustered index.

Hint: Clustered index determines the physical order of records; non-clustered does not.

G13 [M] [Index] [Trap] A table has a unique index built on an attribute that is *not* the field the file is physically sorted by. Is this a primary index or a secondary index?

Hint: The defining test is whether the field orders the file, not whether it is unique.

G14 [H] [B+Tree] [Numerical] A B+ tree leaf can hold at most 3 keys. A leaf currently contains $\{5, 15, 25\}$. Insert key 20 and describe what happens to the leaf and whether the tree's height changes.

Hint: Insertion causes 4 keys $\Rightarrow$ overflow $\Rightarrow$ split; height changes only if the split reaches the root.

15 Guided Practice — Full Worked Solutions

G1 Solution Yes. The S–S lock combination is compatible, so multiple transactions can hold shared locks on the same item simultaneously. This is the whole point of shared locks—they allow concurrent reads without conflict.

G2 Solution Not immediately. Exclusive locks (X) are incompatible with all other locks, including shared locks (S). If T_1 holds X on X, then T_2 's request for S on X is queued and T_2 **waits** until T_1 releases its exclusive lock; T_2 is not automatically aborted.

G3 Solution Yes. Lock conversion is a standard operation in database systems. A transaction can upgrade its shared lock to an exclusive lock when it needs to modify the item, provided no other transaction is concurrently holding a conflicting lock on the same item.

G4 Solution No. Once a transaction releases its first lock (the lock on A), it enters the shrinking phase. In the shrinking phase, it may only release locks—no new locks can be acquired. So the transaction cannot acquire a lock on D.

G5 Solution Strict 2PL: All exclusive locks are held until commit/abort. If T_1 writes X and then T_2 reads that uncommitted write, T_2 must wait for T_1 to commit (or abort) before T_2 can even acquire the lock to read X. This prevents T_2 from reading uncommitted data, which prevents cascading rollback.

Basic 2PL: Allows X locks to be released before commit. So T_1 can write X, release the X lock, and then T_2 can read X—even if T_1 later aborts. If T_1 aborts, T_2 read dirty data and must cascade into rollback.

G6 Solution The four Coffman conditions are:

1. Mutual Exclusion: Each resource can be held by at most one transaction.
2. Hold and Wait: A transaction holding a lock may request another lock.
3. No Preemption: Locks cannot be forcibly taken away.
4. Circular Wait: A cycle of transactions exists, each waiting for a lock held by the next.

All four must be true for deadlock to occur.

G7 Solution Wait-for graph edges: $T_1 \rightarrow T_2$, $T_2 \rightarrow T_3$, $T_3 \rightarrow T_1$. This forms a cycle, and since each of X, Y, Z has a single instance, **a deadlock exists**. One or more transactions must be aborted to break the cycle.

G8 Solution Available = $10 - (3 + 2 + 2) = 3$. T_2 's Need = $2 \leq 3$: run T_2 , Available becomes 5. T_3 's Need = $3 \leq 5$: run T_3 , Available becomes 7. T_1 's Need = $4 \leq 7$: run T_1 . Safe sequence $\langle T_2, T_3, T_1 \rangle$ exists $\Rightarrow$ **state is safe**.

G9 Solution Under the Thomas Write Rule, when $TS(T) < write_TS(X)$, T 's write is simply **ignored/skipped** (treated as an obsolete, already-superseded write) and T is allowed to continue — it is **not** rolled back. Under basic timestamp ordering, this same situation would cause T to be **rolled back**. This is exactly what allows TWR to accept some non-conflict-serializable-but-view-serializable schedules that basic TO would reject.

G10 Solution $bfr = \lfloor 2048/128 \rfloor = 16$ records per block. $b = \lceil 5000/16 \rceil = \lceil 312.5 \rceil = 313$ blocks.

G11 Solution B+ tree is preferred because:

- Internal nodes contain only keys and pointers (no data), making them smaller and allowing more branching (higher fan-out, shallower tree).
- Leaf nodes are linked together, enabling efficient range scans without revisiting internal nodes.
- Better suited to disk-based access patterns, reducing I/O operations.

B-tree has data in internal nodes, which complicates range scans and wastes space in internal nodes.

G12 Solution Clustered Index: The physical order of records in the file matches the index order. There is at most one per table. Records are sorted by the indexed key. Very efficient for range queries on that key.

Non-Clustered Index: Does not dictate physical record order. Multiple non-clustered indexes can exist. The index provides a fast path to find records, but accessing the actual records may require additional lookups (e.g., following a pointer to the data page).

G13 Solution Secondary index. Even though it is unique, the defining property of a primary index is that it is built on the field that **physically orders** the data file. Since this attribute does not order the file, the index is secondary (and must be dense, since the data is not sorted on this field).

G14 Solution Inserting 20 into $\{5, 15, 25\}$ gives $\{5, 15, 20, 25\}$ — 4 keys, overflowing the max of 3. The leaf splits into $\{5, 15\}$ and $\{20, 25\}$, and the key 20 (first key of the right leaf) is copied up to the parent as a separator. **Height does not change** unless this split propagates all the way up and forces the root to split, which is not indicated here (only one leaf split is described).

16 Independent Practice (31 Questions)

Attempt without referring back. Distribution: 5 locks, 5 2PL, 5 deadlock/Banker's/TO, 5 file organization, 5 indexing, 5 B-tree/B+ tree, plus 1 bonus GATE-style B+ tree insertion problem. Answers appear only in the Answer Key.

16.1 Locks (5 questions)

I1 [E] [Locks] [Concept] A shared lock (S) on a data item allows:

1. (A) The transaction to read and write the item.
2. (B) The transaction to read the item and other transactions to read it too.
3. (C) Only one transaction to access the item.
4. (D) The transaction to write the item exclusively.

I2 [E] [Locks] [Concept] An exclusive lock (X) on a data item means:

1. (A) No other transaction can read the item.
2. (B) No other transaction can write the item, but can read it.
3. (C) No other transaction can read or write the item.
4. (D) Multiple transactions can hold X on the item.

I3 [M] [Locks] [Compatibility] Transaction T_1 holds a shared lock on item A. Transaction T_2 now **requests** an exclusive lock on A. What happens?

1. (A) T_2 acquires the X lock immediately; S and X coexist.
2. (B) T_2 's request is queued and T_2 waits until all conflicting S locks on A are released.
3. (C) T_1 is automatically aborted so T_2 can proceed.
4. (D) The system deadlocks immediately.

I4 [M] [Locks] [Conversion] A transaction holds a shared lock on item B. It now wants to write to B. What should it do?

1. (A) Release the S lock and acquire an X lock.
2. (B) Upgrade the S lock to an X lock.
3. (C) Request an additional X lock while keeping S.
4. (D) Cannot write while holding only S; must abort and restart.

I5 [M] [Locks] [Scenario] Transactions T_1 , T_2 , and T_3 all hold shared locks on the same item Z. Is this allowed by the S–X compatibility rule?

1. (A) No; only one transaction can hold a lock on Z.
2. (B) No; S locks conflict with each other.
3. (C) Yes; S locks are mutually compatible.
4. (D) Yes, but only if they acquired the locks in a specific order.

16.2 2PL (5 questions)

I6 [E] [2PL] [Definition] In two-phase locking, the growing phase is:

1. (A) When the transaction grows in size (more records accessed).
2. (B) When the transaction acquires locks without releasing any.
3. (C) When the transaction reads data that has grown (been updated).
4. (D) When the transaction prepares to release locks.

I7 [E] [2PL] [Definition] The shrinking phase in 2PL is:

1. (A) When the transaction releases locks without acquiring new ones.
2. (B) When the transaction shrinks the database (deletes records).
3. (C) When the transaction modifies small amounts of data.
4. (D) When the transaction converts locks from X to S.

I8 [M] [2PL] [Scenario] A transaction following 2PL acquires locks on items A, B, and C. It then releases the lock on A. Later, it tries to acquire a lock on D. Is this allowed by basic 2PL?

1. (A) Yes; 2PL allows any lock acquisition.
2. (B) No; once a lock is released, the transaction cannot acquire new locks.
3. (C) Yes, but only if D was not previously accessed.
4. (D) No; the transaction must acquire all locks upfront.

I9 [M] [2PL] [Comparison] The key difference between basic 2PL and strict 2PL is:

1. (A) Basic 2PL allows any locks; strict 2PL only uses X locks.
2. (B) Basic 2PL can release locks early; strict 2PL holds X locks until commit.
3. (C) Strict 2PL prevents all deadlocks; basic 2PL does not.
4. (D) Basic 2PL uses precedence graphs; strict 2PL does not.

I10 [H] [2PL] [Application] True or False: “If all transactions follow rigorous 2PL, the resulting schedules are guaranteed to be strict schedules.”

1. (A) True.
2. (B) False.

16.3 Deadlock, RAG, Banker's and TO (5 questions)

I11 [E] [Deadlock] [Concept] A deadlock occurs when:

1. (A) Two transactions request the same lock.
2. (B) A transaction waits for a lock held by another transaction.
3. (C) Two or more transactions are in a circular wait for locks.
4. (D) A transaction releases a lock too early.

I12 [M] [Deadlock] [RAG] In a resource-allocation graph where a resource type has **multiple instances**, a cycle is:

1. (A) Both necessary and sufficient for deadlock.
2. (B) Necessary but not sufficient for deadlock.
3. (C) Sufficient but not necessary for deadlock.
4. (D) Neither necessary nor sufficient for deadlock.

I13 [M] [Deadlock] [Prevention] Resource ordering (assigning a total order to resources and requiring transactions to acquire locks in that order) prevents deadlock by eliminating which Coffman condition?

1. (A) Mutual exclusion (B) Hold and wait (C) No preemption (D) Circular wait

I13b [M] [Deadlock] [Wait-Die/Wound-Wait] T_i is **older** than T_j (smaller timestamp). T_i requests a lock currently held by T_j . Under the **wound-wait** scheme, what happens?

1. (A) T_i waits for T_j . (B) T_i is aborted (dies). (C) T_j is forced to abort (wounded), and T_i proceeds. (D) Both transactions are aborted.

I14 [H] [Deadlock] [Banker's] Given Available = 2 instances of a resource, and Needs of $T_1 = 3$, $T_2 = 1$, $T_3 = 4$, which transaction(s) can be granted immediately without exceeding Available?

1. (A) T_1 only (B) T_2 only (C) T_3 only (D) Both T_1 and T_3

I15 [M] [TO] [Concept] Under basic timestamp ordering, if $TS(T) < write_TS(X)$ and T requests to write X , what happens?

1. (A) T 's write is silently ignored. (B) T is rolled back. (C) The write proceeds normally. (D) X is locked for T .

16.4 File Organization (5 questions)

I16 [E] [File Org] [Concept] Which file organization gives the fastest average access for an **equality** search on a large file, assuming no index is used?

1. (A) Heap (B) Sorted with binary search (C) Hash (D) Clustered by an unrelated attribute

I17 [M] [File Org] [Trap] For a query that must read **every** record in a file exactly once (a full scan), which statement is correct?

1. (A) A sorted file is always faster than a heap file for this. (B) A heap file and a sorted file require the same number of block accesses for a full scan. (C) A hash file is fastest for a full scan. (D) Sorting is required before any scan.

I18 [M] [File Org] [Numerical] Block size = 1024 bytes, record size = 64 bytes, unspanned organization. What is the blocking factor?

1. (A) 8 (B) 12 (C) 16 (D) 20

I19 [M] [File Org] [Numerical] Using the blocking factor from I18, how many blocks are needed to store 500 records?

1. (A) 25 (B) 31 (C) 32 (D) 40

I20 [E] [File Org] [Concept] Hash file organization is generally **poor** for:

1. (A) Equality search (B) Range queries (C) Insertion (D) Fixed-length records

16.5 Indexing (5 questions)

I21 [E] [Indexing] [Concept] An index is useful because it:

1. (A) Removes the need for locks.
2. (B) Provides faster access paths to reduce full table scans.
3. (C) Eliminates all redundancy in the database.
4. (D) Automatically prevents all anomalies.

I22 [E] [Indexing] [Classification] A dense index has:

1. (A) Entries only for the first record of each block.
2. (B) An entry for every search-key value in the file.
3. (C) Entries only for the largest values.
4. (D) Multiple levels (multilevel indexing) by definition.

I23 [H] [Indexing] [Trap] A file is physically sorted by `employee_id` (the primary key), and a separate index exists on `department` (non-key, non-ordering). What type is the department index?

1. (A) Primary index (B) Secondary index (C) Clustering index (D) It cannot be indexed

I24 [M] [Indexing] [Classification] A clustering index (as distinct from a primary index) is built on:

1. (A) A key ordering field. (B) A non-key ordering field. (C) A non-ordering key field. (D) A non-ordering non-key field.

I25 [M] [Indexing] [Classification] A clustered index (physical-storage sense) differs from a non-clustered index in that a clustered index:

1. (A) Is always built on the primary key.
2. (B) Dictates the physical order of records in the file.
3. (C) Prevents modifications to records.
4. (D) Can only have one per database (not per table).

16.6 B-Tree / B+ Tree (5 questions)

I26 [M] [B+Tree] [Concept] In a B+ tree, data entries (actual records or record pointers) are stored:

1. (A) Only in internal nodes.
2. (B) Only in leaf nodes.
3. (C) In both internal and leaf nodes.
4. (D) In a separate data file, not in the tree.

I27 [M] [B+Tree] [Concept] A B+ tree of order m (max m children per internal node) has height increase **only** when:

1. (A) Any leaf splits. (B) Any internal node splits. (C) The root splits. (D) The tree becomes unbalanced.

I28 [H] [B+Tree] [Numerical] A leaf holds at most 3 keys and currently contains {2, 8, 14}. After inserting key 11, how many leaves result from this operation, and does the parent necessarily gain a new child?

1. (A) 1 leaf; no change. (B) 2 leaves; parent gains one new child pointer. (C) 3 leaves; parent gains two new child pointers. (D) The insertion is rejected as invalid.

I29 [M] [B-Tree] [Comparison] Compared to a B-tree, a B+ tree of the same block size typically has:

1. (A) Lower fan-out and a taller tree. (B) Higher fan-out and a shorter tree. (C) Identical fan-out. (D) No leaves at all.

I30 [H] [Index] [Trap] True or False: “Every attribute should be indexed to maximize query performance.”

1. (A) True (B) False

16.7 Bonus: GATE-Style B+ Tree Insertion Problem (1 question)

I31 [H] [B+Tree] [GATE PYQ-pattern] [Numerical] A B+ tree has a root with two keys {20, 40} and three children leaves: Leaf A = {10, 15}, Leaf B = {20, 25, 30} (already at its maximum of 3 keys), Leaf C = {40, 50}. Each leaf can hold at most 3 keys, and each internal node can hold at most 3 keys (max 4 children). Key **23** is inserted (it belongs in Leaf B).

1. (A) Leaf B splits into two leaves; the root gains one new key and one new child; tree height stays the same.
2. (B) Leaf B splits, and the root also overflows and splits, so a new root is created and tree height increases by 1.
3. (C) No split occurs; key 23 is simply inserted into Leaf B, which now holds 4 keys.
4. (D) The insertion fails because Leaf B is full and cannot accept a new key.

17 Timed Mini-Test (12 Questions — 15 minutes)

Instructions: Set a timer for 15 minutes. Attempt all 12 without pausing. Answers appear only in the Answer Key.

T1 [E] [Locks] [Concept] Can two transactions hold shared locks on the same item?

1. (A) Yes (B) No (C) Only if they request locks simultaneously (D) Only if one is read-only

T2 [M] [Locks] [Scenario] T_1 holds an exclusive lock on X. T_2 then **requests** a shared lock on X. What happens to T_2 's request?

1. (A) Granted immediately (B) Queued; T_2 waits until T_1 releases the X lock (C) Rejected permanently (D) Granted only if T_2 started first

T3 [E] [2PL] [Definition] After a transaction releases its first lock in 2PL, it enters:

1. (A) Active phase (B) Shrinking phase (C) Lock point (D) Abort phase

T4 [M] [2PL] [Comparison] Strict 2PL differs from basic 2PL in that strict 2PL:

1. (A) Prevents all deadlocks.
2. (B) Holds exclusive locks until commit/abort.
3. (C) Requires acquiring all locks upfront.
4. (D) Uses only X locks, not S locks.

T5 [E] [Deadlock] [Coffman] Which Coffman condition can be eliminated by preemptive lock management?

1. (A) Mutual exclusion (B) No preemption (C) Hold and wait (D) Circular wait

T6 [M] [Deadlock] [Detection] A wait-for graph (single-instance resources) has edges $T_1 \rightarrow T_2$, $T_2 \rightarrow T_3$, $T_3 \rightarrow T_1$. Deadlock status?

1. (A) No deadlock (B) Deadlock exists (C) Unsafe but not deadlock (D) Cannot determine

T7 [E] [Deadlock] [Prevention] Which approach eliminates deadlock by design, not by detection?

1. (A) Detection (B) Avoidance (C) Prevention (D) Recovery

T8 [E] [Index] [Concept] A clustered index determines:

1. (A) The compression method of the file.
2. (B) The physical order of records in the file.
3. (C) The number of levels in the index.
4. (D) The lock strategy for the table.

T9 [M] [B+Tree] [Concept] In a B+ tree, leaf nodes are:

1. (A) Not linked to each other.
2. (B) Linked for efficient range scans.
3. (C) Larger than internal nodes.

4. (D) Unnecessary for search operations.

T10 [E] [Locks] [Trap] An exclusive lock on a data item is compatible with:

1. (A) Another exclusive lock (B) A shared lock (C) No other lock by a different transaction
(D) Any lock

T11 [M] [2PL] [Trap] True or False: “2PL prevents deadlocks.”

1. (A) True (B) False

T12 [H] [Mixed] [Scenario] A transaction follows rigorous 2PL and performs these operations: read A (S lock), write B (X lock), release lock on A. Is this consistent with rigorous 2PL?

1. (A) Yes; S locks can be released early in rigorous 2PL.
2. (B) No; rigorous 2PL requires all locks to be held until commit.
3. (C) Yes; releasing S is allowed anytime.
4. (D) No; the transaction must have released B first.

18 Extended Practice: File Organization, Timestamp Ordering & B+ Tree Numericals (10 Questions)

Use this block if time permits after the main mini-test; these mirror the numerical/reasoning style most commonly seen for this material.

E1 [M] [File Org] [Numerical] Block size = 4096 bytes, record size = 200 bytes, unspanned. Find the blocking factor.

E2 [M] [File Org] [Numerical] Using E1's blocking factor, find the number of blocks for 100,000 records.

E3 [M] [File Org] [Concept] Why can spanned records reduce wasted space compared to unspanned records?

E4 [H] [TO] [Schedule] T_1 has $TS(T_1) = 5$, T_2 has $TS(T_2) = 10$. $write_TS(X) = 10$ already (set by T_2), and $read_TS(X) = 0$ (no transaction has read X yet). T_1 now requests to write X . Under basic TO, what happens? Under TWR, what happens?

E5 [M] [TO] [Concept] Under basic timestamp ordering, what happens to a read request if $TS(T) < write_TS(X)$?

E6 [H] [B+Tree] [Numerical] Internal node fan-out = 50. Estimate the height of a B+ tree indexing 125,000 leaf-level entries (assume leaf capacity also ≈ 50).

E7 [M] [B+Tree] [Concept] Why does a B+ tree generally have a higher fan-out than a B-tree of the same node/block size?

E8 [M] [Deadlock] [RAG] In a single-instance resource-allocation graph, what is both necessary and sufficient for deadlock?

E9 [H] [Banker's] [Numerical] Single resource type, total instances = 6.

Transaction	Allocation	Need
T_1	1	1
T_2	1	2
T_3	2	3

Currently allocated = $1 + 1 + 2 = 4$, so Available = $6 - 4 = 2$. Is this state safe? If so, give a safe completion sequence, showing Available after each step.

E10 [M] [Indexing] [Concept] Why must a secondary index always be dense, while a primary index is typically sparse?

E11 [H] [Indexing] [GATE PYQ-pattern] [Numerical] A file contains $r = 25,000$ records, physically ordered (sorted) by a 15-byte primary key. Each full record is 100 bytes. Block size $B = 1024$ bytes, unspanned organization. A sparse primary index is built on the primary key, with one index entry per data block; each index entry consists of the 15-byte key plus a 5-byte block pointer (20 bytes total). The index is itself sorted and searched using **binary search** on index blocks, where searching b_i index blocks by binary search costs $\lceil \log_2 b_i \rceil$ block accesses, followed by exactly **1** data-block access once the correct data block is identified. Find the total worst-case number of block accesses to locate a given record.

19 Day 1–4 Cumulative Review (8 Rapid-Fire Questions)

Instructions: Maximum 10 minutes, no reteaching. If you miss one, log it and revisit Days 1–4 materials.

C1 [E] [Day 1] [Keys] A candidate key is:

1. (A) A super key that is minimal (B) Any key (C) The primary key (D) A foreign key

C2 [E] [Day 2] [FD/Closure] The closure X^+ is:

1. (A) All attributes reachable from X via FDs (B) All attributes in R (C) All rows with X values (D) The primary key

C3 [M] [Day 2] [2NF] 2NF requires:

1. (A) 1NF + no non-prime partial dependency on composite key (B) 1NF + no redundancy (C) No foreign keys (D) Atomic values

C4 [M] [Day 2] [3NF] 3NF adds to 2NF:

1. (A) No transitive dependency via non-prime (B) All attributes prime (C) No composite keys (D) Recovery capability

C5 [E] [Day 3] [BCNF] BCNF differs from 3NF by:

1. (A) Removing the “or A is prime” exception (B) Being weaker (C) Being the same (D) Requiring weak entities

C6 [M] [Day 4] [ACID] Durability ensures:

1. (A) All-or-nothing (B) Business rules preserved (C) No interference (D) Changes survive crashes

C7 [M] [Day 4] [Serializability] A precedence graph with a cycle indicates:

1. (A) Serializable schedule (B) Not conflict-serializable (C) Recoverable (D) Strict

C8 [E] [Day 4] [Recoverability] A schedule is recoverable if:

1. (A) Conflicts are eliminated (B) Writer commits before reader commits (C) Only committed data read (D) No aborts

Answers: $C1=A$, $C2=A$, $C3=A$, $C4=A$, $C5=A$, $C6=D$, $C7=B$, $C8=B$.

20 Daily Quiz (5 Questions)

Final quick check on Day 5 core concepts. No solutions immediately after; see Answer Key.

Q1 [E] [Locks] [Concept] A shared lock allows:

1. (A) Exclusive access (B) Reading and other transactions can also read (C) Writing only (D) Preemption

Q2 [M] [2PL] [Definition] The lock point in 2PL is:

1. (A) When a transaction starts (B) When all locks are acquired (C) When locks are released (D) When deadlock is detected

Q3 [E] [Deadlock] [Coffman] Eliminate hold-and-wait to prevent deadlock by:

1. (A) Allowing preemption (B) Requiring all locks upfront (C) Ordering resources (D) Detecting cycles

Q4 [M] [Index] [Concept] A B+ tree is preferred for databases because:

1. (A) It eliminates deadlocks (B) Leaf nodes are linked (C) It uses less storage (D) It sorts all values

Q5 [M] [Mixed] [Trap] True or False: “Strict 2PL prevents all deadlocks.”

1. (A) True (B) False

Answers: Q1=B, Q2=B, Q3=B, Q4=B, Q5=B.

21 Complete Answer Key

21.1 Independent Practice Answers

- I1. (B) S locks allow reading and permit other transactions to read concurrently.
- I2. (C) X locks exclude all other access (read or write).
- I3. (B) The request is queued; T_2 waits for the conflicting S lock(s) to be released — it is not auto-aborted.
- I4. (B) Lock conversion (upgrade $S \rightarrow X$) is the standard approach.
- I5. (C) S–S locks are compatible; multiple S locks on the same item are allowed.
- I6. (B) Growing phase = acquire locks, no releases.
- I7. (A) Shrinking phase = release locks, no acquisitions.
- I8. (B) False allowed; once a lock is released, the transaction cannot acquire new locks (2PL rule).
- I9. (B) Strict 2PL holds X locks until commit; basic 2PL does not.
- I10. (A) True. Rigorous 2PL holds all locks until commit/abort, which results in strict schedules.
- I11. (C) Deadlock is a circular wait for locks.
- I12. (B) With multiple instances per resource, a cycle is necessary but not sufficient for deadlock.
- I13. (D) Resource ordering eliminates circular wait.
- I13b. (C) In wound-wait, an **older** requester wounds (preempts) a younger lock-holder, forcing it to abort, and the older transaction proceeds. (Contrast with wait-die: an older requester simply waits for a younger holder; it is the *younger requester* that would die when it requests a lock held by an older transaction.)
- I14. (D) Both T_1 (Need $3 > 2$, actually cannot — recheck) and... *see note below*.
- I15. (B) T is rolled back (a later transaction already wrote X).
- I16. (C) Hash organization gives the fastest average equality search.
- I17. (B) A full scan reads every block exactly once regardless of sort order; heap and sorted files tie here.
- I18. (C) $\lfloor 1024/64 \rfloor = 16$.
- I19. (A) $\lceil 500/16 \rceil = 32$... *see note below*.
- I20. (B) Hashing is poor for range queries.
- I21. (B) Indexes provide faster access paths.
- I22. (B) Dense index has an entry for every search-key value.
- I23. (B) A non-key, non-ordering field gets a secondary index.
- I24. (B) Clustering index = non-key ordering field.
- I25. (B) Clustered index dictates physical record order.
- I26. (B) B+ tree stores data only in leaf nodes.
- I27. (C) Height increases only when the root splits.
- I28. (B) Inserting 11 into $\{2, 8, 14\}$ overflows to $\{2, 8, 11, 14\}$; splits into 2 leaves; parent gains exactly one new child pointer (one new separator key).
- I29. (B) B+ tree internal nodes hold no data, so more keys/pointers fit per block $\Rightarrow$ higher fan-out, shorter tree.
- I30. (B) False. Extra indexes slow inserts/updates; only index frequently searched attributes.
- I31. (A) Inserting 23 into full Leaf $B = \{20, 25, 30\}$ gives $\{20, 23, 25, 30\}$ — 4 keys, overflowing the max of 3. Leaf B splits into $\{20, 23\}$ and $\{25, 30\}$; the separator key **25** (first key of the new right leaf) is copied up into the root. The root currently has 2 keys $\{20, 40\}$ and room for a 3rd (max 3), so it accepts 25 without overflowing: root becomes $\{20, 25, 40\}$ with 4 children (Leaf A, left half of B, right half of B, Leaf C). Since the root itself did not overflow, **no new root is created and height stays the same** — only the leaf level changed. **Correct answer: (A).** Option (B) would only be correct if the root were already full (3 keys) before this insertion, forcing it to split too — always check the parent's current occupancy before concluding height

increases.

Note on I14: Recompute carefully at exam time — with Available = 2, only a transaction whose Need is ≤ 2 can be granted immediately. T_2 (Need = 1) qualifies; T_1 (Need = 3) and T_3 (Need = 4) do not. **Correct answer: (B) T_2 only.**

Note on I19: $\lceil 500/16 \rceil = \lceil 31.25 \rceil = 32$. **Correct answer: (C) 32.**

21.2 Timed Mini-Test Answers

- T1. (A) Yes; S-S locks are compatible.
- T2. (B) Queued; T_2 waits for T_1 to release the X lock.
- T3. (B) Shrinking phase.
- T4. (B) Strict 2PL holds X locks until commit.
- T5. (B) Preemption eliminates the “no preemption” condition.
- T6. (B) The cycle $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ indicates deadlock (single-instance resources).
- T7. (C) Prevention eliminates a Coffman condition by design.
- T8. (B) Clustered index determines physical record order.
- T9. (B) B+ tree leaf nodes are linked for efficient range scans.
- T10. (C) X is incompatible with any other lock by a different transaction.
- T11. (B) False. 2PL ensures serializability but does not prevent deadlocks.
- T12. (B) False. Rigorous 2PL requires all locks (including S) to be held until commit.

21.3 Extended Practice Answers

- E1. $bfr = \lfloor 4096/200 \rfloor = 20$.
- E2. $b = \lceil 100000/20 \rceil = 5000$ blocks.
- E3. Spanned records let a record cross a block boundary, so no space is wasted at the end of a block waiting for a whole extra record to fit; unspanned records leave that leftover space empty.
- E4. First check the read rule for TWR: $TS(T_1) = 5 \geq read_TS(X) = 0$, so T_1 is not rejected on that ground. Basic TO write rule: $TS(T_1) = 5 < write_TS(X) = 10 \Rightarrow T_1$ is rolled back. TWR write rule: since $TS(T_1) \geq read_TS(X)$ but $TS(T_1) < write_TS(X)$, T_1 's write is simply ignored/skipped (treated as an obsolete write already superseded by T_2), and T_1 continues without rollback.
- E5. T is rejected and rolled back — it would be reading a value already overwritten by a later transaction.
- E6. $\log_{50}(125000) \approx 2.97 \Rightarrow \text{height} \approx 3$ (since $50^3 = 125000$).
- E7. Internal nodes carry only keys and pointers (no data entries), so more entries fit per block, giving a higher fan-out than a B-tree, whose internal nodes must also store data.
- E8. A cycle in the wait-for graph (equivalently, in the RAG once resource nodes are collapsed) is both necessary and sufficient for deadlock when every resource type has a single instance.
- E9. Available = 2. T_1 's Need = $1 \leq 2$: run T_1 to completion; it releases its Allocation of 1, so Available = $2 + 1 = 3$. T_2 's Need = $2 \leq 3$: run T_2 ; releases Allocation of 1, Available = $3 + 1 = 4$. T_3 's Need = $3 \leq 4$: run T_3 ; releases Allocation of 2, Available = $4 + 2 = 6$. Safe sequence $\langle T_1, T_2, T_3 \rangle$ exists $\Rightarrow$ **state is safe**.
- E10. A primary index can be sparse (one entry per block) only because the data file is already physically sorted on that key, so all records in a block are guaranteed to be near the indexed anchor value; a secondary index's key does not order the file, so without a dense entry for every value the system cannot know which block a given value lives in.
- E11. **Step 1 — data blocks:** $bfr = \lfloor 1024/100 \rfloor = 10$ records/block; $b = \lceil 25000/10 \rceil = 2500$ data blocks. **Step 2 — index blocks:** one sparse index entry per data block $\Rightarrow 2500$ index entries; index entry size = 20 bytes $\Rightarrow bfr_i = \lfloor 1024/20 \rfloor = 51$ entries/block; $b_i = \lceil 2500/51 \rceil = 50$

index blocks. **Step 3 — binary search cost:** $\lceil \log_2 50 \rceil = 6$ index-block accesses. **Step 4 — total:** 6 (index) + 1 (data block) = **7** block accesses in the worst case.

22 Scoring and Diagnostic Interpretation

Score Band	Interpretation and Next Action
< 60%	Revisit locks (S/X compatibility), 2PL (growing/shrinking phases), and Coffman conditions. Redo Guided Practice G1–G9 before final review.
60 – 75%	Acceptable. Log weak sub-topics in error log. Focus on the distinction between basic/strict/rigorous 2PL and primary vs. secondary in
75 – 85%	Good. Log individual misses, proceed to final review.
> 85%	Strong. Proceed to exam preparation with confidence.

Note: This is a personal study diagnostic only, not an official IOCL or GATE cutoff or scoring benchmark.

23 Revision Checklist

20-Item Revision Checklist — tick before final review

- ☐ I can explain why concurrency control is needed (lost updates, dirty reads, etc.).
- ☐ I know shared locks (S) allow concurrent reads; exclusive locks (X) block all other access.
- ☐ I know the S-X compatibility table: S-S is Yes; everything else is No.
- ☐ I understand lock conversion (upgrade $S \rightarrow X$, downgrade $X \rightarrow S$) and that a conflicting request waits rather than aborting by default.
- ☐ I can identify the growing phase (acquire locks) and shrinking phase (release locks) in 2PL.
- ☐ I know the lock point marks the end of growing phase.
- ☐ I can distinguish basic 2PL, strict 2PL, and rigorous 2PL by lock-release timing.
- ☐ I understand that 2PL guarantees conflict serializability but does NOT prevent deadlocks.
- ☐ I know all four Coffman conditions: mutual exclusion, hold and wait, no preemption, circular wait.
- ☐ I can distinguish deadlock prevention (eliminate Coffman condition), avoidance (check unsafe state), and detection (cycle detection).
- ☐ I understand wait-for graphs/RAGs and know the single-instance-vs-multiple-instance caveat for cycles.
- ☐ I can work a small Banker's algorithm safety check (Allocation, Max, Need, Available, safe sequence).
- ☐ I can state the basic timestamp-ordering read/write rules and the Thomas Write Rule difference.
- ☐ I can compute blocking factor and number of blocks for a file, and know heap vs. sorted vs. hash vs. clustered organization trade-offs.
- ☐ I understand index purposes and can correctly distinguish dense/sparse, primary/secondary (by ordering, not by key-ness), clustering, and clustered/non-clustered.
- ☐ I know B-tree order definitions (max/min children and keys) and can compute them for a given order.
- ☐ I can work a small B+ tree insertion/split numerical and explain when height increases.
- ☐ I can estimate B+ tree height/disk accesses from a given fan-out and record count.
- ☐ I know B+ tree basics: internal nodes guide search, leaf nodes hold data and are linked.
- ☐ I can recognize all 15 common traps and avoid them on the exam.

24 Things I Got Wrong Today

Tag the category before logging specifics:

Question	Topic/Category	My Mistake	Correct Rule

25 One-Page Cheat Sheet

S–X Lock Compatibility

	S	X
S	Yes	No
X	No	No

Conflicting request $\Rightarrow$ waits (not auto-abort).

Basic vs Strict vs Rigorous 2PL

Variant	S Release	X Release	Result
Basic 2PL	Anytime (growing/shrinking)	Anytime (growing/shrinking)	Serializable
Strict 2PL	Anytime	Until commit	Cascadeless, recoverable
Rigorous 2PL	Until commit	Until commit	Strict

Four Coffman Conditions

Mutual Exclusion; Hold and Wait; No Preemption; Circular Wait — all four must be true for deadlock; eliminate any one to prevent it.

Prevention vs. Avoidance vs. Detection

	Prevention	Avoidance	Detection
When	Before	Before	After
Method	Eliminate Coffman	Check safe state (Banker's)	Cycle detection (WFG/RAG)
Cost	Lock/ordering	Simulation	Abort/redo

Wait-for Graph / RAG

Nodes = transactions (WFG) or transactions+resources (RAG). Cycle = deadlock **iff single instance per resource**; with multiple instances, cycle is necessary but not sufficient.

Timestamp Ordering vs. Thomas Write Rule

Read rule (both): reject if $TS(T) < write_TS(X)$. Basic TO write rule: reject/rollback if $TS(T) < read_TS(X)$ or $TS(T) < write_TS(X)$. TWR: same rollback if $TS(T) < read_TS(X)$, but **ignore** (don't rollback) if only $TS(T) < write_TS(X)$. Basic TO $\Rightarrow$ conflict serializable; TWR $\Rightarrow$ view serializable.

File Organization Formulas

$bfr = \lfloor B/R \rfloor$ (blocking factor); $b = \lceil r/bfr \rceil$ (number of blocks). Heap $\approx$ sorted for a full scan; sorted wins for search/range via binary search; hash wins for equality, loses for range; clustered speeds targeted access patterns.

Index Classifications

Dense: entry for every search-key value. Sparse: one entry per block, requires the file to be sorted on that key.

Primary: on the field that **orders** the file (usually the key) — typically sparse. Secondary: on a non-ordering field — always dense. Clustering: non-key ordering field — typically sparse.

Clustered (physical): dictates physical order (at most one per table). Non-clustered: does not (multiple allowed).

B-Tree vs B+ Tree

	B-Tree	B+ Tree
Data in internal nodes	Yes	No
Leaf linked	No	Yes
Range scan	Slower	Fast
Fan-out	Lower	Higher
Height increases	Only on root split	Only on root split
Database use	Rare	Standard

High-Yield Traps

1. S-S is compatible; X conflicts with S and X; conflicting requester **waits**, isn't auto-aborted.
2. 2PL ensures serializability, not deadlock prevention.
3. Basic 2PL $\neq$ strict 2PL (lock-release timing differs).
4. All four Coffman conditions must be present for deadlock.
5. Prevention eliminates Coffman condition; avoidance checks state; detection finds cycles.
6. Unsafe state does not guarantee deadlock (only possible).
7. Wait-for graph cycle = deadlock only for single-instance resources.
8. Wait-for graph (deadlock) $\neq$ precedence graph (serializability).
9. Clustered is a physical property; B+ tree is a data structure.
10. Primary index is defined by ordering the file, not by being "the" key index.
11. Not every attribute needs an index (slows updates).
12. Rigorous 2PL results in strict (not just cascadeless) schedules.
13. Basic TO rolls back stale writes; TWR ignores them instead (view- not conflict-serializable).
14. Heap and sorted files tie for a full sequential scan; sorting only helps search/range.
15. B+ tree height changes only when a split propagates to the root.

End of Day 5 Workbook (Extended / GATE-Compatible Edition) — IOCL Grade-A 2026 CS/IT Preparation

All five days of DBMS theory are now complete.

DAY 5 — SUPPLEMENT

Query Processing, Hashing, Partitioning & Storage

8. Query Processing & Execution

8.1 Query Execution Pipeline

When you submit a query, the DBMS follows this flow:

1. **Parse:** Check SQL syntax and validate table/column names.
2. **Optimize:** Generate candidate execution plans; pick the cheapest (via cost estimation).
3. **Compile:** Translate plan into executable code.
4. **Execute:** Run the plan against actual data; return results.

8.2 Query Execution Plans (QEP)

A QEP is a tree of operators (Scan, Join, Sort, etc.). For example:

`SELECT * FROM Orders o JOIN Customers c ON o.cid = c.id WHERE c.country = 'US'`

Possible Plan 1: SeqScan(Customers) → Filter(country='US') → NestedLoopJoin(Orders) → Result

Possible Plan 2: IndexScan(Customers, idx_country) → Filter(country='US') → HashJoin(Orders) → Result

The optimizer chooses Plan 2 if it estimates lower total cost (fewer disk I/Os, lower CPU).

8.3 Join Algorithms

Nested-Loop Join (NLJ):

- For each outer row, scan entire inner table. Cost: $O(M \times N)$ where M, N are table sizes.
- **Pro:** Works always, minimal memory.
- **Con:** Very slow for large tables. Practically never used for big joins.

Block Nested-Loop Join (BNLJ):

- Load a block (memory buffer) of outer rows; for each block, scan entire inner table once.
- Cost: $O((M/B) \times N)$ where B is block size. Huge improvement over NLJ.
- **Used when:** Tables don't fit in memory or no index available.

Sort-Merge Join (SMJ):

- Sort both tables on join column; then merge in a single pass.
- Cost: $O(M \log M + N \log N)$ assuming both sorted; merge is $O(M + N)$.
- **Pro:** Cache-friendly, efficient if input is already sorted or join order benefits from sort.
- **Con:** Sorting overhead if tables aren't already sorted.

Hash Join:

- Build a hash table on inner table; for each outer row, probe the hash table.
- Cost: $O(M + N)$ average case; handles collisions well.
- **Pro:** Usually fastest for equality joins. Highly parallelizable.
- **Con:** Requires hash table to fit in memory; no good for non-equality or range joins.

8.4 Cost & Cardinality Estimation

The optimizer estimates cost using cardinality (number of rows) at each step:

Selectivity Factor: For a filter WHERE age > 25, the optimizer estimates what fraction of rows pass.

- If column stats show ages uniformly distributed 0–100, selectivity $\approx 75\%$.
- Cardinality estimate: $75\% \times 1,000,000 \text{ rows} = 750,000 \text{ rows}$ passed to next step.

Accuracy matters: If estimate is wrong, optimizer picks a bad plan. Most performance issues come from bad cardinality estimates, not bad algorithms.

9. Hashing & Hash-Based Indexes

9.1 Hash Functions & Static Hashing

A hash function maps keys to bucket addresses. Example: $\text{bucket} = \text{key} \% \text{num_buckets}$

Advantage: $O(1)$ average search time (direct access to bucket).

Problem: If the table grows, the number of buckets is fixed. Collisions increase; performance degrades.

9.2 Dynamic Hashing (Extendible & Linear)

Extendible Hashing:

- Uses a prefix of the hash value as a bucket identifier.
- When a bucket overflows, we **split** it and double the number of buckets (only the overflowing one).
- All other buckets remain unchanged; we just adjust the prefix length.
- **Benefit:** Graceful growth; add buckets incrementally without rehashing all data.

Linear Hashing:

- Similar idea, but buckets are split in a fixed linear order (1, 2, 3, ...).
- When load factor exceeds threshold, split the next bucket in line.
- More uniform memory growth than extendible hashing.

9.3 When to Use Hash Indexes

- **Equality predicates:** WHERE id = 123 (perfect match).
- **NOT good for:** Range queries (WHERE age > 25) or ORDER BY, because hash buckets are unordered.

- **Counterpart to B+ trees:** B+ trees are slower on equality but handle ranges; hash indexes are opposite.

10. Partitioning, Sharding & Replication

10.1 Partitioning (Vertical & Horizontal)

Vertical Partitioning: Split columns across tables/storage. Example: {ID, Name, Email} in one table; {ID, SalaryHistory} in another (sensitive data, separate access).

Horizontal Partitioning: Split rows by some criteria.

- **Range Partitioning:** Orders 2023 in one partition, Orders 2024 in another (split by date).
- **Hash Partitioning:** `customer_id % 4` determines partition (distributes load evenly).
- **List Partitioning:** Region = 'US' → partition 1; Region = 'EU' → partition 2.

Benefit: Faster queries (scan only relevant partition); easier archival/deletion; parallelism.

10.2 Sharding vs Indexing

- **Indexing:** Single table, multiple indexes on different columns. All data remains on same server.
- **Sharding:** Data physically split across multiple servers. Each shard is a separate database.
- **Partitioning (on single DB):** Data split across multiple tables/disks but same logical database.

10.3 Replication

Primary-Replica (Master-Slave): All writes go to primary; replicas are read-only copies (synced asynchronously or synchronously).

Benefit: Read scaling (distribute reads) + availability (failover if primary dies).

Trade-off: Replica lag (temporary inconsistency) in asynchronous replication.

11. RAID: Disk Redundancy & Fault Tolerance

RAID (Redundant Array of Independent Disks) improves reliability and performance by striping/mirroring data across multiple disks.

RAID 0 (Striping): Data split across disks in parallel. $O(n)$ read/write performance boost. **No redundancy**—one disk failure = total loss.

RAID 1 (Mirroring): Exact copy on two disks. Read performance = 1 disk (but can read from either); write = 2 disks. **100% redundancy**—survives one disk failure. Expensive (2× storage).

RAID 5 (Striping with Parity): Data + parity striped across ≥ 3 disks. Can recover from 1 disk

failure. Storage overhead = $1/n$ (for n disks). Most common for servers.

RAID 6 (Dual Parity): Like RAID 5 but with two parity blocks. Survives 2 disk failures. Higher write overhead (calculate 2 parities).

Key for DBMS: RAID 5/6 is standard for production databases. Protects against disk failures (inevitable in large clusters).